

Zadanie 2

Martin Martiš, Michal Škvarka

15. marca 2026

1 Kalibrácia kamery

1.1 Postup riešenia a použité metódy

Na kalibráciu sme použili kameru Ximea a knižnicu OpenCV v jazyku Python. Postup bol nasledovný:

- Pomocou knižnice `xiapi` sme sa pripojili ku kamere a manuálne sme nasníмали 14 fotografií kalibračnej šachovnice (vnútorné rohy 5×7).
- Snímky sme následne previedli do šedotónového formátu a rohy šachovnice sme detegovali pomocou funkcie `cv2.findChessboardCorners`.
- Nájdne rohy sme ešte spresnili na subpixelovej úrovni pomocou funkcie `cv2.cornerSubPix`.
- Potom sme použili funkciu `cv2.calibrateCamera`, ktorá z 3D bodov šachovnice a ich 2D pozícií v obraze vypočítala maticu vnútorných parametrov kamery a distorzné koeficienty.
- Nakoniec sme výsledok overili pomocou funkcie `cv2.undistort`, ktorá odstránila skreslenie šošovky z testovacieho obrázka.

2 Detekcia geometrických tvarov

2.1 Postup riešenia a použité metódy

Program na detekciu základných geometrických tvarov spracováva obraz z kamery v reálnom čase. Najprv sa každý snímok upraví pomocou funkcie `cv2.undistort`, aby sa odstránilo skreslenie spôsobené šošovkou kamery. Potom sa obraz prevedie do šedotónového formátu a mierne sa vyhladí Gaussovým filtrom (`cv2.GaussianBlur`).

Na detekciu jednotlivých tvarov sme využili viacero funkcií z knižnice OpenCV:

- **Kružnice:** Na vyhľadávanie kružníc sme použili Houghovu transformáciu pomocou funkcie `cv2.HoughCircles`.
- **Mnohouholníky:** Ostatné tvary (trojuholník, štvorec, obdĺžnik) sme detegovali pomocou kontúr. Najprv sme našli hrany pomocou algoritmu Canny (`cv2.Canny`) a potom sme z obrázka získali kontúry pomocou funkcie `cv2.findContours`.
- Kontúry sme následne aproximovali polygónom pomocou funkcie `cv2.approxPolyDP`. Tvar objektu sme určili podľa počtu vrcholov:

– 3 vrcholy: trojuholník,

- 4 vrcholy: štvorholník (podľa pomeru strán sme rozlíšili štvorec a obdĺžnik).
- Stred každého detegovaného objektu sme vypočítali pomocou obrazových momentov (`cv2.moments`) a zobrazili ho priamo do obrazu.

3 Bonusová úloha: Určenie veľkosti objektov

3.1 Postup riešenia

V bonusovej úlohe program počíta aj približnú reálnu veľkosť detegovaných objektov v centimetroch. Pri výpočte sme využili výsledky kalibrácie kamery, konkrétne hodnoty ohniskových vzdialeností f_x a f_y .

Predpokladali sme, že objekt sa nachádza vo vzdialenosti 30 cm od kamery. Reálna veľkosť objektu sa potom vypočíta na základe podobnosti trojuholníkov podľa vzťahov:

$$\text{Šírka}_{cm} = \frac{\text{Šírka}_{px} \times \text{Vzdialenosť}_{cm}}{f_x}$$

$$\text{Výška}_{cm} = \frac{\text{Výška}_{px} \times \text{Vzdialenosť}_{cm}}{f_y}$$

Pri kružniciach sme takto počítali priemer z detegovaného polomeru. Pri ostatných tvaroch sme použili ohraničujúci obdĺžnik pomocou funkcie `cv2.boundingRect`. Vypočítané rozmery sa následne zobrazujú priamo do výstupného obrazu spolu s názvom tvaru.

4 Farebný filter

4.1 Postup riešenia a použité metódy

V poslednej časti sme implementovali farebný filter, ktorý dokáže v reálnom čase detegovať vybranú farbu v obraze z kamery a nahradiť ju inou farbou. Postup bol nasledovný:

- Obraz z kamery sme po odstránení skreslenia (`cv2.undistort`) previedli z farebného priestoru BGR do priestoru HSV pomocou funkcie `cv2.cvtColor`. HSV priestor je vhodnejší na prácu s farbami.
- Pre jednoduché nastavovanie filtrov sme vytvorili grafické okno s posuvníkmi (`cv2.createTrackbar`). Pomocou nich bolo možné meniť minimálne a maximálne hodnoty pre zložky H, S a V a zároveň nastaviť novú farbu, ktorou sa majú detegované pixely nahradiť.
- Samotná detekcia farby prebehla pomocou funkcie `cv2.inRange`, ktorá vytvorila binárnu masku vybranej farby.
- Program ošetruje aj prípad, keď rozsah farby prechádza cez hranicu 180 (napríklad pri červenej farbe). Vtedy sa vytvorí dve masky a spoja sa pomocou operácie OR (`cv2.bitwise_or`).
- Na masku sme aplikovali morfológické operácie otvorenia a zatvorenia (`cv2.morphologyEx`), aby sa odstránil šum a vyhladili detegované oblasti.
- Na miestach, kde maska označila vybranú farbu, sme v HSV obraze prepísali hodnotu odtieňa na novú farbu zvolenú posuvníkom.
- Nakoniec sme upravený obraz previedli späť do BGR formátu, aby sa mohol zobraziť.